

R Programming – Full Data Mining & Machine Learning Answers

Includes Apriori, FP-Growth, association rules, Naive Bayes, Decision Tree, regression, ARFF and classifier comparison.

1. Apriori Algorithm and FP-Growth – Support = 3, Confidence = 50%

Use the given market-basket transaction table. Support 3 means an itemset must occur in at least 3 transactions. If there are 5 transactions, support count 3 is $3/5 = 60\%$. Confidence 50% means a rule is accepted when confidence ≥ 0.50 .

R Program

```
library(arules)

# Replace these transactions by the transaction table given in your question
tr <- as(list(
  c("M", "O", "N", "K", "E", "Y"),
  c("D", "O", "N", "K", "E", "Y"),
  c("M", "A", "K", "E"),
  c("M", "U", "C", "K", "Y"),
  c("C", "O", "K", "I", "E")
), "transactions")

# Apriori: support count 3/5, confidence 50%
rules <- apriori(tr,
  parameter=list(supp=3/5, conf=0.50, minlen=2))

inspect(rules)

# Frequent itemsets
freq <- apriori(tr,
  parameter=list(supp=3/5, target="frequent itemsets"))
inspect(freq)
```

Answer / Interpretation: Because the first question refers to an 'above table' that was not included in the message, its exact rules cannot be determined until that table is supplied. The code is ready for the table.

2. Maximum Number of Association Rules

For n distinct items, the maximum number of association rules with a non-empty antecedent and non-empty consequent is $3^n - 2^{n+1} + 1$. This counts all possible rules, even rules having zero support.

R Program

```
max_rules <- function(n) {
  3^n - 2^(n+1) + 1
}

# Example for n = 5
max_rules(5)
```

Answer / Interpretation: For 5 distinct items: $3^5 - 2^6 + 1 = 243 - 64 + 1 = 180$ rules. If the missing table has a different number of distinct items, replace n with that number.

3. Maximum Size of a Frequent Itemset

If there are n distinct items and minimum support is greater than zero, the largest possible frequent itemset can contain all n items, provided all n items occur together at least once.

R Program

```
n <- 5
maximum_size <- n
maximum_size
```

Answer / Interpretation: For $n = 5$, maximum frequent-itemset size = 5. If the table contains n different products, the answer is n .

4. Naive Bayes Classification – Training and Test Data

Split the data into training and testing sets. Train a Naive Bayes classifier on the training set and evaluate predictions on the test set.

R Program

```
# install.packages("e1071")
library(e1071)

data(iris)
set.seed(123)

id <- sample(1:nrow(iris), 0.70*nrow(iris))
train <- iris[id, ]
test <- iris[-id, ]

model <- naiveBayes(Species ~ ., data=train)
pred <- predict(model, test)

print(pred)
print(table(Actual=test$Species, Predicted=pred))

accuracy <- mean(pred == test$Species)
cat("Accuracy =", accuracy*100, "%\n")
```

Answer / Interpretation: The confusion matrix compares actual and predicted classes. Accuracy = correct predictions / total test predictions × 100.

5. Decision Tree Classification – Training and Test Data

Build a classification tree from training data, visualize the tree, predict the test data and calculate accuracy.

R Program

```
# install.packages("rpart")
# install.packages("rpart.plot")
library(rpart)
library(rpart.plot)

data(iris)
set.seed(123)

id <- sample(1:nrow(iris), 0.70*nrow(iris))
train <- iris[id, ]
test <- iris[-id, ]

tree <- rpart(Species ~ ., data=train, method="class")
rpart.plot(tree)

pred <- predict(tree, test, type="class")
print(table(Actual=test$Species, Predicted=pred))

accuracy <- mean(pred == test$Species)
cat("Accuracy =", accuracy*100, "%\n")
```

Answer / Interpretation: The tree shows the decision conditions used to classify observations. Accuracy measures how many test observations were classified correctly.

6. Diabetes.csv – Trend for Different Age Groups

The question asks whether diabetes-related measurements change with age. Use a scatterplot, linear regression and multiple regression. The exact column names must match the CSV file. The common columns are Age, BloodPressure, BMI, Glucose and Outcome.

R Program

```
diabetes <- read.csv("diabetes.csv")

# Inspect the file first
head(diabetes)
names(diabetes)
str(diabetes)

# Scatterplot
plot(diabetes$Age, diabetes$BloodPressure,
     main="Blood Pressure vs Age",
     xlab="Age", ylab="Blood Pressure",
```

```

    pch=19)
abline(lm(BloodPressure ~ Age, data=diabetes))

# Simple linear regression
linear_model <- lm(BloodPressure ~ Age, data=diabetes)
summary(linear_model)

# Multiple regression
multiple_model <- lm(BloodPressure ~ Age + BMI + Glucose,
                    data=diabetes)
summary(multiple_model)

# Diabetes percentage by age group
diabetes$AgeGroup <- cut(diabetes$Age,
                        breaks=c(0,20,30,40,50,60,100),
                        right=FALSE)

age_result <- aggregate(Outcome ~ AgeGroup,
                        data=diabetes, FUN=mean)
age_result$Percent <- age_result$Outcome*100
print(age_result)

barplot(age_result$Percent,
        names.arg=age_result$AgeGroup,
        main="Diabetes Rate by Age Group",
        xlab="Age Group", ylab="Diabetes Rate (%)",
        las=2)

```

Answer / Interpretation: The regression slope indicates the direction of the relationship between age and blood pressure. Multiple regression measures the effect of age while also considering BMI and glucose. The exact trend and numerical coefficients require the actual diabetes.csv file.

7. Given Five Transactions – Apriori, Support = 50%(2), Confidence = 80%

The transactions are T1=(M,O,N,K,E,Y), T2=(D,O,N,K,E,Y), T3=(M,A,K,E), T4=(M,U,C,K,Y), T5=(C,O,K,I,E). There is an inconsistency in the question: 50% of 5 transactions is 2.5, so a literal 50% minimum requires count 3. However, the question explicitly writes '(2)', so many lab sheets intend minimum support count = 2. The solution below follows the stated count 2.

R Program

```

library(arules)

tr <- as(list(
  c("M", "O", "N", "K", "E", "Y"),
  c("D", "O", "N", "K", "E", "Y"),
  c("M", "A", "K", "E"),
  c("M", "U", "C", "K", "Y"),
  c("C", "O", "K", "I", "E")
), "transactions")

# Minimum support count = 2
minsup <- 2/5

freq <- apriori(tr,
  parameter=list(supp=minsup,
                 target="frequent itemsets"))
inspect(freq)

rules <- apriori(tr,
  parameter=list(supp=minsup,
                 conf=0.80,
                 minlen=2))
inspect(sort(rules, by="confidence"))

quality(rules)

```

Answer / Interpretation: Frequent 1-itemsets at count ≥ 2 : C(2), E(4), K(5), M(3), N(2), O(3), Y(3). Frequent 2-itemsets: CK(2), EK(4), EM(2), EN(2), EO(3), EY(2), KM(3), KN(2), KO(3), KY(3), MY(2), NO(2), NY(2), OY(2). Frequent 3-itemsets: EKM(2), EKN(2), EKO(3), EKY(2), ENO(2), ENY(2), EOY(2), KMY(2), KNO(2), KNY(2), KOY(2), NOY(2). Frequent 4-itemsets: EKNO(2), EKNY(2), EKOY(2), ENOY(2), KNOY(2). Frequent 5-itemset: EKNOY(2). R generates the complete rule list satisfying confidence $\geq 80\%$.

8. FP-Growth Tree – Given Five Transactions

FP-Growth first counts items, removes infrequent items and sorts each transaction by descending frequency. The ordered transactions are inserted into a prefix tree. A common prefix is shared by multiple transactions.

R Program

```
items <- list(
  c("M", "O", "N", "K", "E", "Y"),
  c("D", "O", "N", "K", "E", "Y"),
  c("M", "A", "K", "E"),
  c("M", "U", "C", "K", "Y"),
  c("C", "O", "K", "I", "E")
)

frequency <- sort(table(unlist(items)), decreasing=TRUE)
print(frequency)

# Keep items with count >= 2
frequent_items <- names(frequency[frequency >= 2])
print(frequent_items)

# Order every transaction by global frequency
ordered <- lapply(items, function(x) {
  x[x %in% frequent_items][order(
    match(x[x %in% frequent_items], names(frequency))
  )]
})
print(ordered)
```

Answer / Interpretation: The main FP-tree root is NULL. K has the highest frequency (5), followed by E (4), and M/O/Y (3 each), then C/N (2 each). The exact visual tree depends on tie ordering; when two items have the same frequency, either consistent tie order can be used.

9. WEKA-style Preprocessing, Decision Tree and Logistic Regression

The question asks for preprocessing, a tree and logistic classification. R can perform the equivalent analysis using rpart and multinomial logistic regression.

R Program

```
data(iris)

# Preprocess
str(iris)
sum(is.na(iris))
iris$Species <- factor(iris$Species)

set.seed(123)
id <- sample(1:nrow(iris), 0.70*nrow(iris))
train <- iris[id, ]
test <- iris[-id, ]

# Decision Tree
library(rpart)
tree <- rpart(Species ~ ., data=train, method="class")
tree_pred <- predict(tree, test, type="class")
tree_acc <- mean(tree_pred == test$Species)

# Logistic / multinomial logistic regression
# install.packages("nnet")
library(nnet)
logit <- multinom(Species ~ ., data=train, trace=FALSE)
logit_pred <- predict(logit, test)
logit_acc <- mean(logit_pred == test$Species)

cat("Decision Tree Accuracy =", tree_acc*100, "%\n")
cat("Logistic Accuracy =", logit_acc*100, "%\n")
```

Answer / Interpretation: Preprocessing checks structure and missing values. Both models use the same train/test split so their accuracies can be compared fairly.

10. Create an ARFF-style Dataset and Mine Rules

Create a transaction dataset equivalent to an ARFF market-basket dataset. Minimum support = 33.33% and minimum confidence = 60%. A rule is accepted when confidence is at least 60%; otherwise it is rejected.

R Program

```
library(arules)

tr <- as(list(
  c("Bread", "Milk"),
  c("Bread", "Diaper", "Beer", "Eggs"),
  c("Milk", "Diaper", "Beer", "Cola"),
  c("Bread", "Milk", "Diaper", "Beer"),
  c("Bread", "Milk", "Diaper", "Cola"),
  c("Milk", "Diaper", "Beer"),
  c("Bread", "Milk", "Beer"),
  c("Bread", "Diaper", "Cola")
), "transactions")

freq <- apriori(tr,
  parameter=list(supp=0.3333,
    target="frequent itemsets"))
inspect(freq)

rules <- apriori(tr,
  parameter=list(supp=0.3333,
    conf=0.60,
    minlen=2))
inspect(sort(rules, by="confidence"))

q <- quality(rules)
q$Decision <- ifelse(q$confidence >= 0.60,
  "ACCEPTED", "REJECTED")

print(q)
```

Answer / Interpretation: The exact accepted/rejected rules depend on the actual ARFF dataset supplied by the teacher. The code automatically labels every generated rule.

11. Rule-Based Classification vs Decision Tree – Accuracy and Plot

Use the same train/test data for both algorithms. PART is a rule-based classifier available through RWeka; rpart is a decision-tree classifier.

R Program

```
# install.packages("RWeka")
library(RWeka)
library(rpart)

data(iris)
set.seed(123)
id <- sample(1:nrow(iris), 0.70*nrow(iris))
train <- iris[id, ]
test <- iris[-id, ]

# Rule-based classifier
part_model <- PART(Species ~ ., data=train)
part_pred <- predict(part_model, test)
part_accuracy <- mean(part_pred == test$Species)

# Decision tree
tree_model <- rpart(Species ~ ., data=train, method="class")
tree_pred <- predict(tree_model, test, type="class")
tree_accuracy <- mean(tree_pred == test$Species)

print(table(Actual=test$Species, PART=part_pred))
print(table(Actual=test$Species, Tree=tree_pred))

cat("PART Accuracy =", part_accuracy*100, "%\n")
cat("Tree Accuracy =", tree_accuracy*100, "%\n")

accuracy <- c(part_accuracy, tree_accuracy)

barplot(accuracy*100,
  names.arg=c("Rule-Based", "Decision Tree"),
  main="Classification Accuracy Comparison",
  ylab="Accuracy (%)")
```

Answer / Interpretation: The bar chart compares the two test accuracies. The algorithm with the taller bar has higher accuracy for that particular train/test split. Use the same split for a fair comparison.

Important Exam Points

- $\text{Support}(X) = \text{transactions containing } X / \text{total transactions}$.
- $\text{Confidence}(X \rightarrow Y) = \text{support}(X \cup Y) / \text{support}(X)$.
- Apriori principle: every subset of a frequent itemset must also be frequent.
- FP-Growth uses an FP-tree to compress the transaction database and mine frequent patterns.
- Naive Bayes uses Bayes' theorem and predicts the class with the highest posterior probability.
- Decision Tree uses splitting criteria such as Gini impurity or information gain.
- Linear regression uses one predictor; multiple regression uses two or more predictors.
- $\text{Accuracy} = \text{correct predictions} / \text{total test predictions} \times 100$.
- For the five-transaction question, 50% literally means at least 3 transactions, while the worksheet's '(2)' means the intended count threshold is 2. State this assumption in the exam if needed.
- The first Apriori question and the ARFF/diabetes questions require the actual missing datasets for exact numerical outputs.